

5.1. Display Complete Process Memory Layout/Segments and Memory Addresses in C

Program:

```
#include <stdio.h>
#include <stdlib.h>

/* Function declaration */
void function1(void);
void function2(void);

/* DATA SEGMENT */
int globalInt = 100;
float globalFloat = 25.5;
static char globalChar = 'A';

/* BSS SEGMENT */
int uninitializedInt;
float uninitializedFloat;
static int staticUninitialized;

/* RO-DATA */
const char *message = "C Programming";

/* TEXT / CODE SEGMENT */

void function1(void)
{
    printf("Inside function1()\n");
}

void function2(void)
{
    printf("Inside function2()\n");
}

void displayMemory(void)
{
    /* STACK */
    int localInt = 25;
    float localFloat = 12.5;
    char localChar = 'X';

    /* HEAP */
    int *heapInt = malloc(sizeof(int));

    if (heapInt == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    *heapInt = 500;

    printf("    PROCESS MEMORY LAYOUT\n");
```

```

/* STACK */
printf("\nSTACK\n");
printf("localInt    : %p\n", (void *)&localInt);
printf("localFloat   : %p\n", (void *)&localFloat);
printf("localChar    : %p\n", (void *)&localChar);

/* HEAP */
printf("\nHEAP\n");
printf("heapInt      : %p\n", (void *)heapInt);

/* BSS */
printf("\nBSS\n");
printf("uninitializedInt : %p\n", (void *)&uninitializedInt);
printf("uninitializedFloat : %p\n", (void *)&uninitializedFloat);
printf("staticUninitialized: %p\n",
      (void *)&staticUninitialized);

/* DATA */
printf("\nDATA\n");
printf("globalInt    : %p\n", (void *)&globalInt);
printf("globalFloat   : %p\n", (void *)&globalFloat);
printf("globalChar    : %p\n", (void *)&globalChar);

/* RO-DATA */
printf("\nRO-DATA\n");
printf("String Literal : %p\n", (void *)message);

/* TEXT */
printf("\nTEXT / CODE\n");
printf("function1()   : %p\n", (void *)function1);
printf("function2()   : %p\n", (void *)function2);

free(heapInt);
}

int main(void)
{
    displayMemory();

    return 0;
}

```

Observing Stack Frames During Recursive Function Calls (Recursive Stack Analyzer)

```
#include <stdio.h>

long factorial(int n, int call)
{
    int localVariable = n * 10;

    printf("\n-----\n");
    printf("Call Number      : %d\n", call);
    printf("Parameter n       : %d\n", n);
    printf("Parameter Address : %p\n", (void *)&n);
    printf("Local Variable    : %d\n", localVariable);
    printf("Local Address     : %p\n",
        (void *)&localVariable);

    if (n <= 1)
        return 1;

    return n * factorial(n - 1, call + 1);
}

int main(void)
{
    int n;

    printf("Enter number: ");
    scanf("%d", &n);

    if (n < 0)
    {
        printf("Invalid number.\n");
        return 1;
    }

    printf("\nFACTORIAL = %ld\n", factorial(n, 1));

    return 0;
}
```

Heap Memory Lifecycle and Address Tracking

```

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *ptr;
    int i;

    /* malloc() */
    ptr = malloc(3 * sizeof(int));

    if (ptr == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("\nAfter malloc():\n");

    for (i = 0; i < 3; i++)
    {
        printf("ptr[%d] Address = %p\n",
            i, (void *)&ptr[i]);
    }

    /* Initialize */
    ptr[0] = 10;
    ptr[1] = 20;
    ptr[2] = 30;

    printf("\nAfter initialization:\n");

    for (i = 0; i < 3; i++)
    {
        printf("ptr[%d] = %d, Address = %p\n",
            i, ptr[i], (void *)&ptr[i]);
    }

    /* Use */
    printf("\nUsing memory:\n");

    for (i = 0; i < 3; i++)
    {
        printf("Value = %d, Address = %p\n",
            ptr[i], (void *)&ptr[i]);
    }

    /* realloc() */
    ptr = realloc(ptr, 5 * sizeof(int));

    if (ptr == NULL)
    {
        printf("Reallocation failed.\n");
        return 1;
    }
}

```

```

printf("\nAfter realloc():\n");

for (i = 0; i < 3; i++)
{
    printf("ptr[%d] = %d, Address = %p\n",
        i, ptr[i], (void *)&ptr[i]);
}

/* Initialize new elements */
ptr[3] = 40;
ptr[4] = 50;

printf("\nAfter adding new elements:\n");

for (i = 0; i < 5; i++)
{
    printf("ptr[%d] = %d, Address = %p\n",
        i, ptr[i], (void *)&ptr[i]);
}

/* free() */
free(ptr);

printf("\nAfter free():\n");
printf("Heap memory released.\n");

/* Set pointer to NULL */
ptr = NULL;

printf("\nAfter ptr = NULL:\n");
printf("ptr = %p\n", (void *)ptr);

return 0;
}

```